Secure Coding: вариант 04

Бронирование гостиниц

Практическое задание 5. Ручной анализ и Bandit до/после исправлений.

1. Ручной анализ · исходный v04.py

Фрагмент со слайда 4 сохранён без изменения его семи строк. Добавлены импорты и функция booking_fragment(db, password), чтобы return находился внутри функции. Все ссылки ниже относятся к строкам файлов в архиве.

```
 1  import hashlib
 2
 3  from flask import make_response, request
 4
 5
 6  def booking_fragment(db, password):
 7      pw = hashlib.sha256(password.encode()).hexdigest()
 8      code = request.args.get("code", "")              # 8 символов A-Z0-9
 9      sql = f"SELECT room FROM bookings WHERE code = '{code}'"
10      row = db.execute(sql).fetchone()
11      nights = request.form["nights"]                  # ожидается 1..30
12      wish = request.form.get("wish", "")
13      return make_response("<p>Ваше пожелание: " + wish + "</p>")
```

Стр.	Source → sink	CWE / тип	Риск	Рекомендация и строки исправления
9–10	request.args.code → SQL f-строка → db.execute()	CWE-89 SQLi: строковый параметр	High	Передавать code отдельно: WHERE code = ? и (code,). Исправлено: v04_fixed.py:30–31.
12–13	request.form.wish → HTML в make_response()	CWE-79 reflected XSS; HTML-текст	Medium	Экранировать wish в HTML-текстовом контексте с escape(). Исправлено: v04_fixed.py:33.
7	password → hashlib.sha256() → pw	CWE-916 пароль: быстрый хеш без соли	High	Argon2id со случайной солью и затратными параметрами. Отсутствие соли: также CWE-759. Исправлено: v04_fixed.py:9–14, 29.
8	request.args.code → обработка без проверки формата	CWE-20 валидация кода	Medium	Разрешить ровно 8 ASCII-символов A–Z/0–9; иначе HTTP 400. Исправлено: v04_fixed.py:18–20.
11	request.form.nights → строковая переменная без проверки	CWE-20 валидация числа	Medium	Проверить целочисленный ввод и диапазон 1–30; иначе HTTP 400. Исправлено: v04_fixed.py:22–27.

Итого: 5 ручных проблем. Отсутствие соли и быстроту SHA-256 считаем одной проблемой обработки пароля. Проверки code и nights считаем отдельно: это два разных контракта ввода. Фрагмент не показывает сохранение pw или nights в БД.

2. Исправленный фрагмент · v04_fixed.py

```python
import re

from argon2 import PasswordHasher
from argon2.low_level import Type
from flask import abort, make_response, request
from markupsafe import escape


password_hasher = PasswordHasher(
    time_cost=2,
    memory_cost=19456,
    parallelism=1,
    type=Type.ID,
)


def booking_fragment(db, password):
    code = request.args.get("code", "")
    if re.fullmatch(r"[A-Z0-9]{8}", code) is None:
        abort(400, description="Код брони: ровно 8 символов A-Z0-9.")

    nights_text = request.form.get("nights", "")
    if re.fullmatch(r"[0-9]{1,2}", nights_text) is None:
        abort(400, description="Количество ночей: целое число от 1 до 30.")
    nights = int(nights_text)
    if not 1 <= nights <= 30:
        abort(400, description="Количество ночей: целое число от 1 до 30.")

    pw = password_hasher.hash(password)
    sql = "SELECT room FROM bookings WHERE code = ?"
    row = db.execute(sql, (code,)).fetchone()
    wish = request.form.get("wish", "")
    return make_response(f"<p>Ваше пожелание: {escape(wish)}</p>")
```

Как устранены проблемы

SQLi: SQL-строка постоянна, code передаётся параметром SQLite. Проверка формата code дополняет параметризацию.

XSS: escape(wish) кодирует специальные HTML-символы; пожелание остаётся текстом внутри <p>. Пользовательский HTML не исполняется.

Пароль: Argon2id, memory_cost=19456 KiB (19 MiB), time_cost=2, parallelism=1. PasswordHasher создаёт случайную соль; тип, параметры и соль входят в результирующую строку хеша. SHA-256 не считается сломанным криптографическим примитивом, но прямое быстрое хеширование без соли не подходит для хранения паролей.

Ввод: fullmatch проверяет code целиком. Для nights допускаются только ASCII-цифры, затем выполняются int() и проверка 1 ≤ nights ≤ 30. Неверные и отсутствующие значения отклоняются с HTTP 400 до SQL-запроса.

Оба файла — фрагменты обработчика, а не полноценное приложение. db — SQLite-совместимое соединение; request требует контекста Flask. Регистрация, вход и сохранение бронирований не входят в приведённый фрагмент.

3. Статический анализ · Bandit 1.9.4

Проверены именно v04.py и v04_fixed.py на Python 3.12.14. JSON- и HTML-отчёты получены реальными запусками и приложены в архиве.

Показатель	До: v04.py	После: v04_fixed.py
Число находок	1	0
Правила	B608: hardcoded_sql_expressions	—
Severity / Confidence	Medium / Low	—
CWE / расположение	CWE-89; ./v04.py:9	—
Код выхода (JSON и HTML)	1: найдена проблема	0: находок нет
Ошибки / подавления nosec	0 / 0	0 / 0

TP / FP / FN: сопоставление с ручным анализом

Проблема	Кто нашёл	Итог	До	После	Комментарий
SQLi v04.py:9–10	Bandit B608 + вручную	TP	1 находка	0 находок	Данные из args.code реально попадают в SQL. Параметризация: fixed:30–31.
Reflected XSS v04.py:12–13	Вручную	FN	Есть; Bandit не сообщил	Устранено вручную проверенным escape()	Bandit не обнаружил этот путь form.wish → HTML. Исправление: fixed:33.
SHA-256 пароля v04.py:7	Вручную	FN	Есть; Bandit не сообщил	Argon2id со случайной солью	B324 не сообщает о SHA-256; пропущено неподходящее применение к паролю. fixed:9–14, 29.
Формат code v04.py:8	Вручную	FN	Не проверен	8 ASCII A–Z/0–9	Сканер не знает контракт кода брони. Исправление: fixed:18–20.
Диапазон nights v04.py:11	Вручную	FN	Не проверен	Целое 1–30	Сканер не знает бизнес-ограничение. Исправление: fixed:22–27.

Итог: вручную 5 проблем; у Bandit до исправлений 1 находка; **TP = 1, FP = 0, FN = 4**. Единственная находка B608 подтверждена источником пользовательских данных. Все четыре FN относятся к состоянию «до»; после исправления их устранение проверено отдельно. Ни одно правило не подавлялось.

Low Confidence отражает ограничение правила Bandit, а не безопасность кода. В ручном анализе SQLi оценена как High по возможному воздействию; Severity сканера сохранена без изменения.

4. Воспроизведение и вывод

Команды выполняются в каталоге распакованного архива. Зависимости: Flask 3.1.3, MarkupSafe 3.0.4, argon2-cffi 25.1.0, Bandit 1.9.4 (requirements.txt). Команды ниже приведены для Linux/macOS.

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/bandit v04.py -f json -o reports/bandit_before.json
.venv/bin/bandit v04.py -f html -o reports/bandit_before.html
.venv/bin/bandit v04_fixed.py -f json -o reports/bandit_after.json
.venv/bin/bandit v04_fixed.py -f html -o reports/bandit_after.html
```

Для каждого запуска «до» ожидается exit code 1, а для «после» — 0. Код 1 не означает сбой анализатора: он сообщает о находке. Каталог reports уже присутствует в архиве. JSON — для проверки результатов, HTML — для удобного просмотра.

Проверка исправлений

Выполнены 9 функциональных проверок с контекстом запроса Flask и SQLite в памяти; все завершились успешно. В исходном фрагменте воспроизведены изменение SQL-запроса вводом и отражение активного HTML. В исправленном проверены корректный поиск, параметризация, отказ для неверного code и nights, отсутствие nights, границы 1 и 30, экранирование HTML, разные соли Argon2id и проверка правильного/неправильного пароля.

Вывод (4 предложения)

Bandit автоматически обнаружил построение SQL через f-строку и помог проверить устранение этой находки после параметризации. Ручной анализ дополнительно выявил отражённый XSS, неподходящую обработку пароля и отсутствие двух проверок ввода. Эти пропуски показывают, что анализатор не всегда учитывает контекст HTML, назначение криптографической операции и бизнес-правила. Поэтому нулевой отчёт после исправлений следует дополнять ручным ревью и функциональной проверкой защит.

Состав сдачи

PDF: SSAD_Week5_Variant_04.pdf — единый отчёт.
Архив: SSAD_Week5_Variant_04_code.zip — исходный и исправленный фрагменты, зависимости и фактические отчёты анализатора.

```
v04.py
v04_fixed.py
requirements.txt
reports/bandit_before.json
reports/bandit_before.html
reports/bandit_after.json
reports/bandit_after.html
reports/scan_exit_codes.json
```

Основание: SSAD_Week5_Practical_Secure_Coding.pptx, слайды 4 (вариант), 9 и 13 (ручной анализ), 15–19 и 22–23 (Bandit и triage), 24 (состав сдачи). Выполнена обязательная часть; SonarQube — бонус.